Task 1: Payment System

Requirements

- Create three classes: **CreditCard**, **UPI**, **Cash**.
- Each class should contain `pay(amount)`.
- The method should display a different payment message for each payment type.
- Create objects and call the *pay()* method.

Concept

Polymorphism

Program

```
# Task 1: Payment System (Polymorphism)
class CreditCard:
    def pay(self, amount):
        print(f"Paid Rs.{amount} using Credit Card")


class UPI:
    def pay(self, amount):
        print(f"Paid Rs.{amount} using UPI")


class Cash:
    def pay(self, amount):
        print(f"Paid Rs.{amount} in Cash")


card = CreditCard()
upi = UPI()
cash = Cash()

card.pay(5000)
upi.pay(1200)
cash.pay(300)
```

Output

```
Paid Rs.5000 using Credit Card
Paid Rs.1200 using UPI
Paid Rs.300 in Cash
```

Explanation

All three classes have a method with the same name, *pay()*, but each one behaves differently. The same method call produces a different message depending on the object it is called on. This is polymorphism.

Task 2: Animal Sounds

Requirements

- Create the classes **Dog**, **Cat**, **Cow**, **Lion**.
- Each class should contain `sound()`.
- Create a function `make_sound(animal)` that calls *sound()*.
- Pass objects of all four classes to the function.

Concept

Polymorphism

Program

```
# Task 2: Animal Sounds (Polymorphism)
class Dog:
    def sound(self):
        print("Dog barks: Woof Woof")


class Cat:
    def sound(self):
        print("Cat meows: Meow Meow")


class Cow:
    def sound(self):
        print("Cow moos: Moo Moo")


class Lion:
    def sound(self):
        print("Lion roars: Roar")


def make_sound(animal):
    animal.sound()


make_sound(Dog())
make_sound(Cat())
make_sound(Cow())
make_sound(Lion())
```

Output

```
Dog barks: Woof Woof
Cat meows: Meow Meow
Cow moos: Moo Moo
Lion roars: Roar
```

Explanation

make_sound() does not care which animal it receives. It simply calls *animal.sound()*, and Python runs the version of *sound()* that belongs to that object. One function works with many different object types.

Task 3: Notification System

Requirements

- Create three classes: **Email**, **SMS**, **WhatsApp**.
- Each class should contain `send(message)`.
- Create a function `notify(obj)` that calls *send()*.
- Pass objects of all three classes to the function.
- **Condition:** Do not use *isinstance()* or *type()*.

Concept

Duck Typing

Program

```
# Task 3: Notification System (Duck Typing)
# The function never checks the class of the object it receives.
class Email:
    def send(self, message):
        print("Sending Email:", message)


class SMS:
    def send(self, message):
        print("Sending SMS:", message)


class WhatsApp:
    def send(self, message):
        print("Sending WhatsApp message:", message)


def notify(obj):
    obj.send("Your order has been shipped")


notify(Email())
notify(SMS())
notify(WhatsApp())
```

Output

```
Sending Email: Your order has been shipped
Sending SMS: Your order has been shipped
Sending WhatsApp message: Your order has been shipped
```

Explanation

Duck typing means "if it has a *send()* method, it can be used". *notify()* never checks the type of the object. It just calls *obj.send()*, and it works for any object that has that method, even though Email, SMS and WhatsApp are not related by inheritance. No *isinstance()* or *type()* is used.

Task 4: Vehicle System

Requirements

- Create three classes: **Car**, **Bike**, **Bus**.
- Each class should contain `start()`.
- Create a function `start_vehicle(vehicle)` that calls *start()*.
- Use the function with all three objects.

Concept

Duck Typing

Program

```
# Task 4: Vehicle System (Duck Typing)
class Car:
    def start(self):
        print("Car engine started")


class Bike:
    def start(self):
        print("Bike engine started")


class Bus:
    def start(self):
        print("Bus engine started")


def start_vehicle(vehicle):
    vehicle.start()


start_vehicle(Car())
start_vehicle(Bike())
start_vehicle(Bus())
```

Output

```
Car engine started
Bike engine started
Bus engine started
```

Explanation

The classes are independent and share no parent class. *start_vehicle()* only needs the object to have a *start()* method. If an object behaves like a vehicle (it can start), it is accepted.

Task 5: Employee Salary

Requirements

- Create a parent class **Employee** with the method `calculate_salary()`.
- Create the child classes **Developer**, **Manager**, **Tester**.
- Override *calculate_salary()* in every child class and provide different salary calculations.

Concept

Method Overriding

Program

```
# Task 5: Employee Salary (Method Overriding)
class Employee:
    def calculate_salary(self):
        print("Salary is not defined for a general employee")


class Developer(Employee):
    def calculate_salary(self):
        basic = 50000
        bonus = 10000
        print("Developer salary:", basic + bonus)


class Manager(Employee):
    def calculate_salary(self):
        basic = 70000
        allowance = 15000
        print("Manager salary:", basic + allowance)


class Tester(Employee):
    def calculate_salary(self):
        basic = 40000
        incentive = 5000
        print("Tester salary:", basic + incentive)


Employee().calculate_salary()
Developer().calculate_salary()
Manager().calculate_salary()
Tester().calculate_salary()
```

Output

```
Salary is not defined for a general employee
Developer salary: 60000
Manager salary: 85000
Tester salary: 45000
```

Explanation

Each child class redefines *calculate_salary()* with its own calculation (basic + bonus, basic + allowance, basic + incentive). When the method is called on a child object, Python runs the child's version instead of the parent's. The Employee object runs the parent's version.

Task 6: Bank Account

Requirements

- Create a parent class **BankAccount** with the method `interest()`.
- Create the child classes **SavingsAccount**, **CurrentAccount**.
- Override the *interest()* method in both child classes.
- Create objects and call the method.

Concept

Method Overriding

Program

```
# Task 6: Bank Account (Method Overriding)
class BankAccount:
    def interest(self):
        print("Interest rate is not defined for a general account")


class SavingsAccount(BankAccount):
    def interest(self):
        balance = 100000
        rate = 4
        print("Savings Account interest:", balance * rate / 100)


class CurrentAccount(BankAccount):
    def interest(self):
        balance = 100000
        rate = 0
        print("Current Account interest:", balance * rate / 100)


BankAccount().interest()
SavingsAccount().interest()
CurrentAccount().interest()
```

Output

```
Interest rate is not defined for a general account
Savings Account interest: 4000.0
Current Account interest: 0.0
```

Explanation

SavingsAccount overrides *interest()* with a 4% rate (100000 x 4 / 100 = 4000.0), while CurrentAccount overrides it with a 0% rate. Both replace the parent's default version of the method.

Task 7: Food Ordering System

Requirements

- Create the class structure: **Food** with children **Pizza**, **Burger**, **Biryani**.
- The parent class should contain `prepare()`.
- Override *prepare()* in every child class.
- Create a function `order(food)` that calls *food.prepare()*.
- Pass different food objects to the *order()* function.

Concept

Inheritance, Method Overriding, Polymorphism

Program

```
# Task 7: Food Ordering System (Inheritance + Overriding + Polymorphism)
class Food:
    def prepare(self):
        print("Preparing food")


class Pizza(Food):
    def prepare(self):
        print("Preparing Pizza: making dough, adding cheese, baking")


class Burger(Food):
    def prepare(self):
        print("Preparing Burger: grilling patty, adding veggies")


class Biryani(Food):
    def prepare(self):
        print("Preparing Biryani: cooking rice with spices")


def order(food):
    food.prepare()


order(Pizza())
order(Burger())
order(Biryani())
```

Output

```
Preparing Pizza: making dough, adding cheese, baking
Preparing Burger: grilling patty, adding veggies
Preparing Biryani: cooking rice with spices
```

Explanation

Pizza, Burger and Biryani **inherit** from Food and **override** *prepare()*. The function *order()* accepts any Food object and calls *prepare()*, and the correct child version runs at runtime. This is **polymorphism**.

Task 8: Notification System

Requirements

- Create the class structure: **Notification** with children **EmailNotification**, **SMSNotification**, **PushNotification**.
- Each child class should override `send()`.
- Create `send_notification(notification)` that calls *notification.send()*.
- Call the function with all three objects.
- **Condition:** Do not use *type()* or *isinstance()*.

Concept

Inheritance, Method Overriding, Polymorphism, Duck Typing

Program

```
# Task 8: Notification System (Inheritance + Overriding + Polymorphism + Duck Typing)
# The function never checks the class of the object it receives.
class Notification:
    def send(self):
        print("Sending notification")


class EmailNotification(Notification):
    def send(self):
        print("Email notification sent")


class SMSNotification(Notification):
    def send(self):
        print("SMS notification sent")


class PushNotification(Notification):
    def send(self):
        print("Push notification sent")


def send_notification(notification):
    notification.send()


send_notification(EmailNotification())
send_notification(SMSNotification())
send_notification(PushNotification())
```

Output

```
Email notification sent
SMS notification sent
Push notification sent
```

Explanation

The three child classes inherit from Notification and override *send()*. *send_notification()* only calls *notification.send()* and never checks the object's type, so it relies on duck typing, and the overridden method of each child is chosen at runtime (polymorphism).